ORBIT · PYTHON · PAPER 05

第 5 套

60 题 · 完整题目与答案

单选 30 多选 15 填空 / 简答 15

本套分节

一、单项选择题（30） 二、不定项选择题（15） 三、填空题（5） 四、程序代码阅读题（5） 五、程序设计题（5）

答案与解析紧随每题；本文件用于复习核对，建议先独立作答。

一、单项选择题

01 • 单项选择

关于Python的组合（Composition），以下说法正确的是：

A. 组合是通过继承实现的
B. 组合是通过包含其他类的实例实现的
C. 组合不如继承灵活
D. Python不支持组合

答案 B. 组合是通过包含其他类的实例实现的

解析 组合（Composition）是通过在一个类中包含其他类的实例来实现功能复用，而不是通过继承。

示例：

```
# 组合方式
class Engine:
    def start(self):
        return "引擎启动"
    def stop(self):
        return "引擎停止"
class Wheels:
    def rotate(self):
        return "轮子转动"
class Car:
    def __init__(self):
        # 组合：包含其他类的实例
        self.engine = Engine()
        self.wheels = Wheels()
    def start(self):
        return self.engine.start()
    def drive(self):
        return f"{self.engine.start()}, {self.wheels.rotate()}"
car = Car()
print(car.start())
# 引擎启动
print(car.drive())
# 引擎启动, 轮子转动
知识拓展：组合vs继承
# 继承方式（不够灵活）
class Vehicle:
    def move(self):
        return "移动"
class Car(Vehicle):
    def drive(self):
        return "开车"
# 组合方式（更灵活）
class Engine:
    def run(self):
        return "引擎运行"
class ElectricMotor:
    def run(self):
```

```
        return "电机运行"
class ModernCar:
    def __init__(self, power_source):
        self.power = power_source
    # 可以灵活替换
    def drive(self):
        return self.power.run()
# 灵活组合
gas_car = ModernCar(Engine())
electric_car = ModernCar(ElectricMotor())
print(gas_car.drive())
# 引擎运行
print(electric_car.drive()) # 电机运行
```

02 • 单项选择

以下代码的输出是什么？

```
class Engine:
    def start(self):
        return "引擎启动"
class Car:
    def __init__(self):
        self.engine = Engine()
    def start(self):
        return self.engine.start()
car = Car()
print(car.start())
```

A. Car启动
B. 引擎启动
C. None
D. 报错

答案 B. 引擎启动

解析 Car类通过组合包含Engine实例，调用 engine.start() 返回"引擎启动"。

示例：

```
class Engine:
    def start(self):
        return "引擎启动"
class Car:
    def __init__(self):
        self.engine = Engine()
    # 组合
    def start(self):
        return self.engine.start()
# 委托给engine
car = Car()
print(car.start())
# 引擎启动
```

知识拓展：组合的优势

```
# 多个组件组合
class Engine:
```

```
    def start(self):
        return "引擎启动"
class Radio:
    def play(self):
        return "播放音乐"
class GPS:
    def navigate(self):
        return "导航中"
class ModernCar:
    def __init__(self):
        self.engine = Engine()
        self.radio = Radio()
        self.gps = GPS()
    def start_journey(self):
        actions = [
            self.engine.start(),
            self.radio.play(),
            self.gps.navigate()
        ]
        return ", ".join(actions)
car = ModernCar()
print(car.start_journey())
# 引擎启动, 播放音乐, 导航中
```

03 • 单项选择

关于 __getattr__ 和 __getattribute__ 的区别，以下说法正确的是：

A. 完全相同

B. __getattr__ 在属性不存在时调用， __getattribute__ 总是调用

C. __getattribute__ 在属性不存在时调用

D. 两者都不常用

答案 B. __getattr__ 在属性不存在时调用， __getattribute__ 总是调用

解析 __getattr__ 只在属性不存在时调用， __getattribute__ 在访问任何属性时都会调用。

示例：

```
class GetAttrExample:
    def __init__(self):
        self.x = 10
    def __getattr__(self, name):
        # 只在属性不存在时调用
        print(f"__getattr__ called for: {name}")
        return f"默认值 for {name}"
obj1 = GetAttrExample()
print(obj1.x)
# 10 - 直接访问，不调用__getattr__
print(obj1.y)
# __getattr__ called for: y\n默认值 for y
class GetAttributeExample:
    def __init__(self):
        # 必须用object.__setattr__避免无限递归
        object.__setattr__(self, 'x', 10)
```

```
    def __getattribute__(self, name):
        # 访问任何属性都调用
        print(f"__getattribute__ called for: {name}")
        return object.__getattribute__(self, name)
obj2 = GetAttributeExample()
print(obj2.x)
# __getattribute__ called for: x\n10
print(obj2.y)
# __getattribute__ called for: y\nAttributeError
知识拓展：避免无限递归
class SafeExample:
    def __init__(self):
        self.data = {}
    def __getattr__(self, name):
        # 安全：只处理不存在的属性
        return self.data.get(name, None)
    def __setattr__(self, name, value):
        # 安全：使用__dict__
        if name == 'data':
            object.__setattr__(self, name, value)
        else:
            self.data[name] = value
obj = SafeExample()
obj.x = 100
print(obj.x)
# 100
print(obj.y)
# None
```

04 • 单项选择

以下哪个魔法方法用于实现对象的哈希值？

A. __hash__()

B. __id__()

C. __key__()

D. __digest__()

答案 A. __hash__()

解析 __hash__() 魔法方法用于计算对象的哈希值，使对象可以用作字典键或集合元素。

示例：

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __hash__(self):
        # 计算哈希值
        return hash((self.x, self.y))
    def __eq__(self, other):
        # 必须同时实现__eq__
        return self.x == other.x and self.y == other.y
p1 = Point(1, 2)
p2 = Point(1, 2)
```

```
p3 = Point(3, 4)
# 可以用作字典键
point_dict = {p1: "第一个点", p3: "第三个点"}
print(point_dict[p2])
# 第一个点 - p1和p2相等
# 可以用于集合
point_set = {p1, p2, p3}
print(len(point_set))
# 2 - p1和p2被认为相同
知识拓展：哈希的规则
# 规则：如果a == b，则hash(a) == hash(b)
class BadHash:
    def __init__(self, value):
        self.value = value
    def __hash__(self):
        return 1
    # 总是返回相同的哈希值（低效但合法）
    def __eq__(self, other):
        return self.value == other.value
# 不可变对象才应该可哈希
class ImmutablePoint:
    def __init__(self, x, y):
        self._x = x
        self._y = y
    @property
    def x(self):
        return self._x
    @property
    def y(self):
        return self._y
    def __hash__(self):
        return hash((self._x, self._y))
    def __eq__(self, other):
        return self._x == other.x and self._y == other.y
```

05 • 单项选择

关于Python的 `__slots__` ，以下说法正确的是：

A. 用于限制实例属性

B. 可以节省内存

C. 使用后不能动态添加属性

D. 以上都正确

答案 D. 以上都正确

解析 所有选项都是上下文管理器的正确特性。

示例：

```
import time
# A. 使用with语句
class Timer:
    def __enter__(self):
        self.start = time.time()
        return self
```

```
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.end = time.time()
        print(f"耗时: {self.end - self.start:.2f}秒")
        return False
# 不抑制异常
with Timer():
    sum([i**2 for i in range(1000000)])
# 输出：耗时: 0.XX秒
# B. __enter__和__exit__
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode
        self.file = None
    def __enter__(self):
        self.file = open(self.filename, self.mode)
        return self.file
    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.file:
            self.file.close()
        return False
# C. 自动资源管理
with FileManager('test.txt', 'w') as f:
    f.write('Hello')
# 文件自动关闭
# D. 可以处理异常
class ErrorHandler:
    def __enter__(self):
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            print(f"捕获异常: {exc_type.__name__}: {exc_val}")
            return True
        # 抑制异常
        return False
with ErrorHandler():
    raise ValueError("测试错误")
print("继续执行")
# 异常被抑制，继续执行
知识拓展：contextlib简化上下文管理器
from contextlib import contextmanager
@contextmanager
def timer():
    start = time.time()
    try:
        yield
    finally:
        end = time.time()
        print(f"耗时: {end - start:.2f}秒")
with timer():
    sum([i**2 for i in range(1000000)])
```

06 • 单项选择

以下代码的输出是什么？

```
class Point:
    __slots__ = ['x', 'y']
    def __init__(self, x, y):
        self.x = x
        self.y = y
p = Point(1, 2)
p.z = 3
```

A. 正常执行

B. 抛出AttributeError

C. z被成功添加

D. 输出3

答案 B. 抛出AttributeError

解析 使用 __slots__ 后，类实例不能动态添加新属性，会抛出 AttributeError 。

示例：

```
# 普通类：可以动态添加属性
class NormalClass:
    def __init__(self, x):
        self.x = x
obj1 = NormalClass(10)
obj1.y = 20
# 可以添加新属性
print(obj1.y)
# 20
# 使用__slots__：不能动态添加属性
class SlotClass:
    __slots__ = ['x']
    # 只允许x属性
    def __init__(self, x):
        self.x = x
obj2 = SlotClass(10)
try:
    obj2.y = 20
# 尝试添加新属性
except AttributeError as e:
    print(f"错误: {e}")
# 错误: 'SlotClass' object has no attribute 'y'
知识拓展： __slots__ 的优势
import sys
class WithoutSlots:
    def __init__(self, x, y):
        self.x = x
        self.y = y
class WithSlots:
    __slots__ = ['x', 'y']
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
obj1 = WithoutSlots(1, 2)
obj2 = WithSlots(1, 2)
print(f"Without slots: {sys.getsizeof(obj1.__dict__)} bytes")
print(f"With slots: {sys.getsizeof(obj2)} bytes")
# __slots__节省内存，访问速度更快
```

07 • 单项选择

关于生成器的 throw() 方法，以下说法正确的是：

A. 用于向生成器抛出异常
B. 用于终止生成器
C. 用于发送值
D. 与 send() 相同

答案 A. 用于向生成器抛出异常

解析 throw() 方法用于向生成器抛出异常。

示例：

```
def generator_with_exception():
    try:
        yield 1
        yield 2
        yield 3
    except ValueError as e:
        print(f"捕获异常: {e}")
        yield "异常处理后"
gen = generator_with_exception()
print(next(gen))
# 1
print(gen.throw(ValueError, "测试异常"))
# 捕获异常: 测试异常\n异常处理后
知识拓展：生成器的三个控制方法
def controllable_gen():
    try:
        while True:
            value = yield
            print(f"接收到: {value}")
    except GeneratorExit:
        print("生成器关闭")
gen = controllable_gen()
next(gen)
# 启动
gen.send(10)
# send(): 发送值
gen.throw(ValueError)
# throw(): 抛出异常
gen.close()
# close(): 关闭生成器
```

08 • 单项选择

以下代码的输出是什么？

```
def gen():
```

```
    try:
        yield 1
    except ValueError:
        yield 2
    yield 3
g = gen()
print(next(g))
print(g.throw(ValueError))
print(next(g))
```

A. 1 2 3

B. 1 2 StopIteration

C. 1 3 StopIteration

D. 报错

答案 A. 1 2 3

解析 输出1，抛出ValueError被except捕获返回2，最后finally块返回3。

示例：

```
def test_function():
    try:
        print(1)
        raise ValueError("错误")
        print("不会执行")
    # 不会执行
    except ValueError:
        print(2)
        return "from except"
    finally:
        print(3)
        return "from finally"
# 覆盖except的return
result = test_function()
print(f"返回值: {result}")
# 输出：
# 1
# 2
# 3
# 返回值: from finally
```

知识拓展：finally中return的风险

```
# 不推荐在finally中return
def risky_function():
    try:
        return "try value"
    finally:
        return "finally value"
# 覆盖try的返回值
print(risky_function())
# finally value - 可能导致混淆
```

09 • 单项选择

关于协程（Coroutine），以下说法正确的是：

A. 协程是特殊的生成器
B. 使用 async def 定义
C. 使用 await 调用
D. 以上都正确

答案 D. 以上都正确

解析 所有选项都是协程的正确特性（尽管协程是生成器的高级应用）。

示例：

```
# A. 使用yield表达式
def simple_coroutine():
    while True:
        x = yield
        # 接收值
        print(f"接收到: {x}")
coro = simple_coroutine()
next(coro)
# 启动
coro.send(10)
# 接收到: 10
# B. 可以发送值
def accumulator():
    total = 0
    while True:
        value = yield total
        if value is not None:
            total += value
acc = accumulator()
next(acc)
# 启动
print(acc.send(10))
# 10
print(acc.send(20))
# 30
# C. 可以接收异常
def error_handling_coro():
    while True:
        try:
            x = yield
            print(f"处理: {x}")
        except ValueError:
            print("处理异常")
coro = error_handling_coro()
next(coro)
coro.send(10)
# 处理: 10
coro.throw(ValueError)
# 处理异常
# D. 实现状态机
def state_machine():
    state = "初始"
    while True:
```

```
        event = yield state
        if state == "初始" and event == "开始":
            state = "运行中"
        elif state == "运行中" and event == "停止":
            state = "已停止"
sm = state_machine()
print(next(sm))
# 初始
print(sm.send("开始"))
# 运行中
print(sm.send("停止"))
# 已停止
```

10 • 单项选择

以下代码的输出是什么？

```
class MyIter:
    def __init__(self, n):
        self.n = n
        self.i = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.i < self.n:
            self.i += 1
            return self.i
        raise StopIteration
it = MyIter(3)
print(list(it))
```

A. [1, 2, 3] [1, 2, 3]
B. [1, 2, 3] []
C. [] []
D. 报错

答案 B. [1, 2, 3] []

解析 迭代器只能遍历一次，第一次迭代后耗尽，第二次返回空列表。

```
示例：
data = [1, 2, 3]
iter_data = iter(data)
# 第一次迭代
result1 = list(iter_data)
print(result1)
# [1, 2, 3]
# 第二次迭代（迭代器已耗尽）
result2 = list(iter_data)
print(result2)
# []
知识拓展：可重复迭代vs一次性迭代
# 列表：可重复迭代
data_list = [1, 2, 3]
```

```
print(list(data_list))
# [1, 2, 3]
print(list(data_list))
# [1, 2, 3] - 可以重复
# 迭代器：一次性
data_iter = iter([1, 2, 3])
print(list(data_iter))
# [1, 2, 3]
print(list(data_iter))
# [] - 已耗尽
# 生成器：一次性
data_gen = (x for x in [1, 2, 3])
print(list(data_gen))
# [1, 2, 3]
print(list(data_gen))
# [] - 已耗尽
# 创建可重复迭代的类
class ReusableIterator:
    def __init__(self, data):
        self.data = data
    def __iter__(self):
        return iter(self.data)
# 每次返回新迭代器
reusable = ReusableIterator([1, 2, 3])
print(list(reusable))
# [1, 2, 3]
print(list(reusable))
# [1, 2, 3] - 可以重复
```

11 • 单项选择

关于装饰器类，以下说法正确的是：

A. 装饰器只能是函数
B. 类也可以作为装饰器
C. 类装饰器需要实现 `__call__()`
D. B和C都正确

答案 D. B和C都正确

解析 类也可以作为装饰器，需要实现 `__call__()` 方法。B和C都正确。

示例：

```
class Counter:
    def __init__(self, func):
        self.func = func
        self.count = 0
    def __call__(self, *args, **kwargs):
        self.count += 1
        print(f"调用次数: {self.count}")
        return self.func(*args, **kwargs)
@Counter
def greet(name):
    return f"Hello, {name}"
print(greet("Alice"))
```

```
print(greet("Bob"))
print(f"总调用次数: {greet.count}")
知识拓展：类装饰器 vs 函数装饰器
# 函数装饰器
def func_decorator(func):
    def wrapper(*args):
        print("函数装饰器")
        return func(*args)
    return wrapper
# 类装饰器
class ClassDecorator:
    def __init__(self, func):
        self.func = func
    def __call__(self, *args):
        print("类装饰器")
        return self.func(*args)
@func_decorator
def func1():
    return "Function"
@ClassDecorator
def func2():
    return "Class"
print(func1())
print(func2())
```

12 • 单项选择

以下代码的输出是什么？

```
class Counter:
    def __init__(self, func):
        self.func = func
        self.count = 0
    def __call__(self, *args):
        self.count += 1
        return self.func(*args)
@Counter
def say_hello():
    return "Hello"
say_hello()
print(say_hello.count)
```

A. 0

B. 1

C. 2

D. 3

答案 C. 2

解析 @Counter 装饰器记录函数调用次数，调用两次后 count 为2。

示例：

```
class Counter:
    def __init__(self, func):
```

```
        self.func = func
        self.count = 0
    def __call__(self, *args):
        self.count += 1
        return self.func(*args)
@Counter
def say_hello():
    return "Hello"
say_hello()
# count = 1
say_hello()
# count = 2
print(say_hello.count)
# 2
知识拓展：带状态的装饰器
class CallLogger:
    def __init__(self, func):
        self.func = func
        self.calls = []
    def __call__(self, *args, **kwargs):
        import time
        start = time.time()
        result = self.func(*args, **kwargs)
        end = time.time()
        self.calls.append({
            'args': args,
            'kwargs': kwargs,
            'result': result,
            'duration': end - start
        })
        return result
    def get_stats(self):
        return {
            'total_calls': len(self.calls),
            'avg_duration': sum(c['duration'] for c in self.calls) / len(self.calls) if se
        }
@CallLogger
def add(a, b):
    return a + b
add(1, 2)
add(3, 4)
print(add.get_stats())
```

13 • 单项选择

关于 contextlib 模块，以下说法正确的是：

A. 提供上下文管理器工具

B. @contextmanager 装饰器可以简化上下文管理器的创建

C. 使用 yield 分隔 __enter__ 和 __exit__

D. 以上都正确

答案 D. 以上都正确

解析 contextlib 模块提供上下文管理器工具，所有选项都正确。

示例：

```
from contextlib import contextmanager
@contextmanager
def file_manager(filename, mode):
    print(f"打开文件: {filename}")
    f = open(filename, mode)
    try:
        yield f
    # __enter__和__exit__的分隔点
    finally:
        print(f"关闭文件: {filename}")
        f.close()
with file_manager('test.txt', 'w') as f:
    f.write("Hello, World!")
```

知识拓展：多种上下文管理器实现方式

```
from contextlib import contextmanager, closing
import sqlite3
# 1. 类实现
class DatabaseConnection:
    def __init__(self, db_name):
        self.db_name = db_name
    def __enter__(self):
        self.conn = sqlite3.connect(self.db_name)
        return self.conn
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.conn.close()
# 2. @contextmanager装饰器
@contextmanager
def database_connection(db_name):
    conn = sqlite3.connect(db_name)
    try:
        yield conn
    finally:
        conn.close()
# 3. closing()辅助函数
with closing(sqlite3.connect(':memory:')) as conn:
    cursor = conn.cursor()
```

14 • 单项选择

以下代码的输出是什么？

```
def func(x, y, z=3):
    return x + y + z
args = (1, 2)
kwargs = {'z': 4}
print(func(*args, **kwargs))
```

A. 6

B. 7

C. 10

D. 报错

答案 B. 7

解析 *args 解包为 (1, 2)， **kwargs 解包为 {'z': 4}，结果为1+2+4=7。

示例：

```
def func(x, y, z=3):
    return x + y + z
args = (1, 2)
kwargs = {'z': 4}
result = func(*args, **kwargs)
print(result)
# 7
知识拓展：参数解包的各种场景
# 场景1: 位置参数和关键字参数混合
def process(a, b, c=10, d=20):
    return a + b + c + d
args = (1, 2)
kwargs = {'c': 30, 'd': 40}
print(process(*args, **kwargs))
# 73
# 场景2: 字典和列表解包
data = {'name': 'Alice', 'age': 25}
info = ['Bob', 30]
def show_info(name, age):
    print(f"{name}: {age}岁")
show_info(**data)
# Alice: 25岁
show_info(*info)
# Bob: 30岁
# 场景3: 合并字典
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, 'd': 4}
merged = {**dict1, **dict2}
print(merged)
# {'a': 1, 'b': 2, 'c': 3, 'd': 4}
# 场景4: 合并列表
list1 = [1, 2, 3]
list2 = [4, 5, 6]
merged = [*list1, *list2]
print(merged)
# [1, 2, 3, 4, 5, 6]
```

15 • 单项选择

关于Python的参数注解（Type Hints），以下说法正确的是：

A. 强制类型检查

B. 只是提示，不强制

C. 运行时会检查类型

D. Python 2就支持

答案 B. 只是提示，不强制

解析 Python的类型注解只是提示，不强制类型检查。

示例：

```
def add(x: int, y: int) -> int:
    return x + y
# 即使传入错误类型，也能正常运行
result = add("Hello", "World")
print(result)
# HelloWorld
print(type(result))
# <class 'str'>
```

知识拓展：类型注解的使用

```
from typing import List, Dict, Optional, Union, Callable
# 基本类型注解
def greet(name: str) -> str:
    return f"Hello, {name}"
# 容器类型注解
def process_list(items: List[int]) -> int:
    return sum(items)
# 可选类型
def find_user(user_id: int) -> Optional[Dict[str, str]]:
    return {'name': 'Alice'} if user_id == 1 else None
# 联合类型
def process(data: Union[int, str]) -> str:
    return str(data)
# 函数类型
def apply(func: Callable[[int, int], int], x: int, y: int) -> int:
    return func(x, y)
# 使用mypy进行静态类型检查
# pip install mypy
# mypy script.py
```

16 • 单项选择

Pandas中，以下哪个方法用于数据分箱？

A. cut()

B. qcut()

C. bin()

D. A和B都对

答案 D. A和B都对

解析 Pandas中 cut() 和 qcut() 都用于数据分箱，前者等宽分箱，后者等频分箱。

示例：

```
import pandas as pd
import numpy as np
data = pd.Series([1, 7, 5, 4, 6, 8, 3, 2, 9, 10])
# cut(): 等宽分箱
bins_cut = pd.cut(data, bins=3)
print("等宽分箱:")
print(bins_cut.value_counts().sort_index())
# qcut(): 等频分箱
bins_qcut = pd.qcut(data, q=3)
print("\n等频分箱:")
print(bins_qcut.value_counts().sort_index())
```

知识拓展：分箱的高级用法

```
import pandas as pd
data = pd.Series([18, 25, 35, 45, 55, 65, 75])
# 自定义分箱边界
bins = [0, 30, 60, 100]
labels = ['青年', '中年', '老年']
result = pd.cut(data, bins=bins, labels=labels)
print(result)
# 包含右边界
result = pd.cut(data, bins=3, right=True)
print(result)
# 包含左边界
result = pd.cut(data, bins=3, right=False)
print(result)
# 返回分箱索引
result = pd.cut(data, bins=3, labels=False)
print(result)
```

17 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
s = pd.Series([1, 2, 3, 4, 5])
result = pd.cut(s, bins=2)
print(len(result.cat.categories))
```

A. 1

B. 2

C. 3

D. 5

答案 B. 2

解析 pd.cut(s, bins=2) 将数据分成2个区间。

示例：

```
import pandas as pd
s = pd.Series([1, 2, 3, 4, 5])
result = pd.cut(s, bins=2)
print(result)
print(f"\n区间数量: {len(result.cat.categories)}")
```

知识拓展：分类数据操作

```
import pandas as pd
s = pd.Series([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
cut_result = pd.cut(s, bins=3)
print("类别:", cut_result.cat.categories)
print("编码:", cut_result.cat.codes)
print("值计数:", cut_result.value_counts())
```

18 • 单项选择

Pandas中， MultiIndex 的作用是：

A. 多列索引

B. 多级索引

C. 多个DataFrame

D. 多种数据类型

答案 B. 多级索引

解析 MultiIndex 用于创建多级索引（hierarchical indexing）。

示例：

```
import pandas as pd
# 创建多级索引
arrays = [
    ['A', 'A', 'B', 'B'],
    [1, 2, 1, 2]
]
index = pd.MultiIndex.from_arrays(arrays, names=['字母', '数字'])
df = pd.DataFrame({'值': [10, 20, 30, 40]}, index=index)
print(df)
# 多级索引访问
print("\n访问A:")
print(df.loc['A'])
知识拓展：多级索引的高级操作
import pandas as pd
# 方式1: from_tuples
index = pd.MultiIndex.from_tuples([('A', 1), ('A', 2), ('B', 1), ('B', 2)])
# 方式2: from_product
index = pd.MultiIndex.from_product([['A', 'B'], [1, 2]])
# 方式3: from_frame
df_index = pd.DataFrame({'字母': ['A', 'A', 'B'], '数字': [1, 2, 1]})
index = pd.MultiIndex.from_frame(df_index)
# 多级索引的stack和unstack
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
}, index=['x', 'y', 'z'])
stacked = df.stack()
print(stacked)
print(stacked.unstack())
```

19 • 单项选择

关于Pandas的 stack() 和 unstack() ，以下说法正确的是：

A. stack() 将列转为行

B. unstack() 将行转为列

C. 两者互为逆操作

D. 以上都正确

答案 D. 以上都正确

解析 stack() 将列转为行， unstack() 将行转为列，两者互为逆操作。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2],
    'B': [3, 4]
}, index=['x', 'y'])
print("原始DataFrame:")
```

```
print(df)
stacked = df.stack()
print("\nstack()后:")
print(stacked)
unstacked = stacked.unstack()
print("\nunstack()后:")
print(unstacked)
知识拓展：数据重塑详解
import pandas as pd
# 复杂的stack/unstack
df = pd.DataFrame({
    '城市': ['北京', '北京', '上海', '上海'],
    '季度': ['Q1', 'Q2', 'Q1', 'Q2'],
    '销售额': [100, 150, 120, 180]
})
# pivot
pivoted = df.pivot(index='城市', columns='季度', values='销售额')
print(pivoted)
# melt (逆操作)
melted = pivoted.reset_index().melt(id_vars='城市', var_name='季度', value_name='销售额')
print(melted)
```

20 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
result = df.T
print(result.shape)
```

A. (2, 2)

B. (4,)

C. (2, 2)

D. (1, 4)

答案 A. (2, 2)

解析 DataFrame转置后，行列互换，(2, 2)转置后仍是(2, 2)。

```
示例：
import pandas as pd
df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
print("原始shape:", df.shape)
# (2, 2)
result = df.T
print("转置后shape:", result.shape)
# (2, 2)
print("\n转置后的DataFrame:")
print(result)
知识拓展：转置的各种场景
import pandas as pd
# 不同形状的转置
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
print(f"原始: {df1.shape}")
```

```
# (3, 2)
print(f"转置: {df1.T.shape}")
# (2, 3)
# 带索引的转置
df2 = pd.DataFrame(
    {'Math': [90, 85], 'English': [88, 92]},
    index=['Alice', 'Bob']
)
print("\n原始:")
print(df2)
print("\n转置后:")
print(df2.T)
```

21 • 单项选择

pathlib.Path 对象的 as_posix() 方法的作用是：

A. 转换为POSIX风格路径
B. 转换为Windows路径
C. 检查是否是POSIX路径
D. 创建POSIX路径

答案 A. 转换为POSIX风格路径

解析 as_posix() 将路径转换为POSIX风格（使用正斜杠 / ）。

示例：

```
from pathlib import Path
# Windows风格路径
p = Path('C:\\Users\\test')
print(f"原始: {p}")
print(f"POSIX: {p.as_posix()}")
# C:/Users/test
```

知识拓展：路径格式转换

```
from pathlib import Path
import os
# 不同平台的路径处理
if os.name == 'nt':
    # Windows
    p = Path('C:\\Users\\test\\file.txt')
else:
    # Unix/Linux/Mac
    p = Path('/home/test/file.txt')
print(f"原始路径: {p}")
print(f"POSIX格式: {p.as_posix()}")
print(f"字符串格式: {str(p)}")
print(f"URI格式: {p.as_uri()}")
```

22 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('C:\\Users\\test')
print(p.as_posix())
```

A. C:\Users\test

B. C:/Users/test
C. /Users/test
D. 报错

答案 B. C:/Users/test

解析 as_posix() 将 C:\Users\test 转换为 C:/Users/test 。
示例：

```
from pathlib import Path
p = Path('C:\\Users\\test')
print(p.as_posix())
# C:/Users/test
```

23 • 单项选择

Path.cwd() 方法返回的是：

A. 用户主目录
B. 当前工作目录
C. 根目录
D. 脚本所在目录

答案 B. 当前工作目录

解析 Path.cwd() 返回当前工作目录（Current Working Directory）。
示例：

```
from pathlib import Path
print("当前工作目录:", Path.cwd())
print("用户主目录:", Path.home())
```

24 • 单项选择

关于 Path.relative_to() 方法，以下说法正确的是：

A. 计算相对路径
B. 转换为相对路径
C. 两个路径必须有共同的父目录
D. 以上都正确

答案 D. 以上都正确

解析 relative_to() 用于计算相对路径，所有选项都正确。
示例：

```
from pathlib import Path
p1 = Path('/a/b/c/d')
p2 = Path('/a/b')
relative = p1.relative_to(p2)
print(relative)
# c/d
知识拓展：相对路径计算
from pathlib import Path
# 计算相对路径
base = Path('/home/user/projects')
target = Path('/home/user/projects/myapp/src/main.py')
relative = target.relative_to(base)
print(relative)
# myapp/src/main.py
```

```
# 不共享父目录会报错
try:
    p1 = Path('/home/user')
    p2 = Path('/var/log')
    p1.relative_to(p2)
except ValueError as e:
    print(f"错误: {e}")
```

25 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p1 = Path('/a/b/c')
p2 = Path('/a')
print(p1.relative_to(p2))
```

A. /a/b/c
B. b/c
C. c
D. 报错

答案 B. b/c

解析 /a/b/c 相对于 /a 的路径是 b/c 。

示例：

```
from pathlib import Path
p1 = Path('/a/b/c')
p2 = Path('/a')
print(p1.relative_to(p2))
# b/c
```

26 • 单项选择

关于 collections 模块的 defaultdict ，以下说法正确的是：

A. 访问不存在的键会返回默认值
B. 需要指定默认值工厂函数
C. 比普通字典更高效
D. A和B都正确

答案 D. A和B都正确

解析 defaultdict 访问不存在的键会返回默认值，需要指定默认值工厂函数。A和B都正确。

示例：

```
from collections import defaultdict
# int作为默认工厂
d = defaultdict(int)
d['a'] += 1
print(d['b'])
# 0
# list作为默认工厂
d = defaultdict(list)
d['key'].append(1)
print(d['key'])
# [1]
```

知识拓展：defaultdict的各种用法

```
from collections import defaultdict
# 分组
items = [('fruit', 'apple'), ('vegetable', 'carrot'),
         ('fruit', 'banana'), ('vegetable', 'lettuce')]
groups = defaultdict(list)
for category, item in items:
    groups[category].append(item)
print(dict(groups))
# 计数
text = "hello world"
char_count = defaultdict(int)
for char in text:
    char_count[char] += 1
print(dict(char_count))
# 嵌套defaultdict
nested = defaultdict(lambda: defaultdict(int))
nested['a']['x'] = 1
nested['a']['y'] = 2
print(dict(nested))
```

27 • 单项选择

以下代码的输出是什么？

```
from collections import defaultdict
d = defaultdict(int)
d['a'] += 1
d['b'] += 2
print(d['c'])
```

A. 0

B. None

C. KeyError

D. 1

答案 A. 0

解析 defaultdict(int) 的默认值为0，访问不存在的键 'c' 返回0。

示例：

```
from collections import defaultdict
d = defaultdict(int)
d['a'] += 1
d['b'] += 2
print(d['c'])
# 0
print(d)
# defaultdict(<class 'int'>, {'a': 1, 'b': 2, 'c': 0})
```

28 • 单项选择

关于 collections.Counter ，以下说法正确的是：

A. 用于计数

B. 继承自字典

C. 提供 most_common() 方法

D. 以上都正确

答案 D. 以上都正确

解析 Counter 用于计数，继承自字典，提供 most_common() 方法，所有选项都正确。

示例：

```
from collections import Counter
c = Counter('abracadabra')
print(c)
# Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})
print(c.most_common(2))
# [('a', 5), ('b', 2)]
知识拓展：Counter的高级用法
from collections import Counter
# 基本计数
words = ['apple', 'banana', 'apple', 'cherry', 'banana', 'apple']
c = Counter(words)
print(c.most_common(2))
# [('apple', 3), ('banana', 2)]
# 算术运算
c1 = Counter(['a', 'b', 'c'])
c2 = Counter(['b', 'c', 'd'])
print(c1 + c2)
# Counter({'b': 2, 'c': 2, 'a': 1, 'd': 1})
print(c1 - c2)
# Counter({'a': 1})
print(c1 & c2)
# Counter({'b': 1, 'c': 1})
print(c1 | c2)
# Counter({'a': 1, 'b': 1, 'c': 1, 'd': 1})
# 更新计数
c = Counter()
c.update('abracadabra')
c.update({'a': 1, 'b': 2})
print(c)
```

29 • 单项选择

以下代码的输出是什么？

```
from collections import Counter
c = Counter('abracadabra')
print(c.most_common(1))
```

A. [('a', 5)]

B. [('b', 2)]

C. 'a'

D. 5

答案 A. [('a', 5)]

解析 'a'在'abracadabra'中出现5次，是最多的，返回 [('a', 5)] 。

示例：

```
from collections import Counter
c = Counter('abracadabra')
print(c.most_common(1))
```

```
# [('a', 5)]
print(c.most_common(3))
# [('a', 5), ('b', 2), ('r', 2)]
```

30 • 单项选择

关于 itertools.chain() ，以下说法正确的是：

A. 用于连接多个迭代器
B. 返回新列表
C. 只能连接两个迭代器
D. 会复制数据

答案 A. 用于连接多个迭代器

解析 itertools.chain() 用于连接多个迭代器，返回迭代器不是列表，不复制数据。

示例：

```
import itertools
list1 = [1, 2, 3]
list2 = [4, 5, 6]
list3 = [7, 8, 9]
# 连接多个迭代器
chained = itertools.chain(list1, list2, list3)
print(list(chained))
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
知识拓展：chain的高级用法
import itertools
# chain.from_iterable: 从可迭代对象的可迭代对象中连接
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = list(itertools.chain.from_iterable(matrix))
print(flattened)
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
# 与生成器配合
def get_numbers():
    yield from range(1, 4)
def get_letters():
    yield from 'abc'
combined = itertools.chain(get_numbers(), get_letters())
for item in combined:
    print(item, end=' ')
# 1 2 3 a b c
```

二、不定项选择题

31 • 不定项选择

关于Python的设计模式，以下说法正确的有：

A. 单例模式可以用 __new__() 实现
B. 工厂模式可以用类方法实现
C. 观察者模式可以用装饰器实现
D. 策略模式可以用函数实现

答案 A. 单例模式可以用 __new__() 实现；B. 工厂模式可以用类方法实现；C. 观察者模式可以用装饰器实现；D. 策略模式可以用函数实现

解析 所有选项都是组合优于继承的原因正确

32 • 不定项选择

以下哪些是Python的容器协议方法：

A. __len__()
B. __getitem__()
C. __setitem__()
D. __contains__()

答案 A. __len__()；B. __getitem__()；C. __setitem__()；D. __contains__()

解析 所有选项都是容器协议方法

33 • 不定项选择

关于Python的弱引用，以下说法正确的有：

A. 使用 weakref 模块
B. 不会增加引用计数
C. 可以避免循环引用
D. 所有对象都支持弱引用

答案 A. 使用 weakref 模块；B. 不会增加引用计数；C. 可以避免循环引用

解析 不是所有对象都支持弱引用

34 • 不定项选择

关于生成器的性能优势，以下说法正确的有：

A. 节省内存
B. 惰性计算
C. 适合处理大数据
D. 总是比列表快

答案 A. 节省内存；B. 惰性计算；C. 适合处理大数据

解析 生成器不总是比列表快

35 • 不定项选择

以下哪些是迭代器的应用场景：

A. 读取大文件
B. 无限序列
C. 数据流处理
D. 节省内存

答案 A. 读取大文件；B. 无限序列；C. 数据流处理；D. 节省内存

解析 所有选项都是迭代器的应用场景

36 • 不定项选择

关于装饰器的最佳实践，以下说法正确的有：

A. 使用 functools.wraps 保留元数据

B. 装饰器应该是幂等的
C. 避免在装饰器中修改全局状态
D. 装饰器可以有副作用

答案 A. 使用 functools.wraps 保留元数据；B. 装饰器应该是幂等的；C. 避免在装饰器中修改全局状态

解析 装饰器最佳实践包括使用 functools.wraps 保留元数据、避免修改全局状态，但装饰器可以有副作用（如日志记录）。
（详细示例可参考第1-4套装饰器相关题目）

37 • 不定项选择

关于闭包和装饰器的关系，以下说法正确的有：

A. 装饰器通常使用闭包实现
B. 闭包可以保存装饰器的状态
C. 所有装饰器都是闭包
D. 闭包是装饰器的基础

答案 A. 装饰器通常使用闭包实现；B. 闭包可以保存装饰器的状态；D. 闭包是装饰器的基础

解析 装饰器通常使用闭包实现，闭包是装饰器的基础，但不是所有装饰器都是闭包（类装饰器除外）。
（详细示例可参考第1-4套闭包和装饰器相关题目）

38 • 不定项选择

以下哪些是Pandas的窗口函数：

A. rolling()
B. expanding()
C. ewm()
D. sliding()

答案 A. rolling()；B. expanding()；C. ewm()

解析 Pandas的窗口函数包括 rolling() 、 expanding() 、 ewm() ，没有 sliding() 。
（详细示例可参考第1-4套Pandas相关题目）

39 • 不定项选择

关于Pandas的性能优化，以下说法正确的有：

A. 使用向量化操作
B. 避免循环遍历DataFrame
C. 使用 apply() 比循环快
D. 选择合适的数据类型

答案 A. 使用向量化操作；B. 避免循环遍历DataFrame；C. 使用 apply() 比循环快；D. 选择合适的数据类型

解析 Pandas性能优化的所有选项都正确：使用向量化操作、避免循环、使用apply()、选择合适的数据类型。
（详细示例可参考第1-4套Pandas性能优化相关题目）

40 • 不定项选择

Pandas中，以下哪些方法可以用于数据导出：

A. to_csv()
B. to_excel()

C. to_json()
D. to_sql()

答案 A. to_csv()；B. to_excel()；C. to_json()；D. to_sql()

解析 所有选项都是Pandas的数据导出方法。
（详细示例可参考第1-4套Pandas相关题目）

41 • 不定项选择

关于 pathlib 的优势，以下说法正确的有：

A. 面向对象
B. 跨平台
C. 链式调用
D. 更安全

答案 A. 面向对象；B. 跨平台；C. 链式调用；D. 更安全

解析 pathlib 的所有优势选项都正确：面向对象、跨平台、链式调用、更安全。
（详细示例可参考第1-4套pathlib相关题目）

42 • 不定项选择

Path 对象的以下哪些方法返回布尔值：

A. exists()
B. is_file()
C. is_dir()
D. is_absolute()

答案 A. exists()；B. is_file()；C. is_dir()；D. is_absolute()

解析 所有选项都是返回布尔值的Path方法。
（详细示例可参考第1-4套pathlib相关题目）

43 • 不定项选择

关于Python的推导式，以下说法正确的有：

A. 列表推导式 []
B. 字典推导式 {}
C. 集合推导式 {}
D. 元组推导式 ()

答案 A. 列表推导式 []；B. 字典推导式 {}；C. 集合推导式 {}

解析 元组推导式使用 () 实际上是生成器表达式，不是元组推导式。
（详细示例可参考第1-4套推导式相关题目）

44 • 不定项选择

以下哪些是Python的函数式编程工具：

A. map()
B. filter()
C. reduce()
D. lambda

答案 A. map()；B. filter()；C. reduce()；D. lambda

解析 所有选项都是Python的函数式编程工具。
（详细示例可参考第1-4套函数式编程相关题目）

45 • 不定项选择

关于Python的性能优化，以下说法正确的有：

A. 使用生成器代替列表
B. 使用局部变量代替全局变量
C. 使用 __slots__ 减少内存
D. 使用列表推导式代替循环

答案 A. 使用生成器代替列表；B. 使用局部变量代替全局变量；C. 使用 __slots__ 减少内存；D. 使用列表推导式代替循环

解析 所有选项都是Python性能优化的有效方法。
（详细示例可参考第1-4套性能优化相关题目）

三、填空题

46 • 填空 / 简答

Python中，使用 _____ 可以实现单例模式，使用 _____ 可以实现工厂模式。

答案 yield ， send()

解析 协程的双向通信机制。
示例：

```
def accumulator():
    total = 0
    while True:
        value = yield total
        # 返回total，接收value
        if value is not None:
            total += value
coro = accumulator()
print(next(coro))
# 0 - 启动协程
print(coro.send(10))
# 10
print(coro.send(5))
# 15
print(coro.send(3))
# 18
```

知识拓展：协程是Python异步编程的基础。

47 • 填空 / 简答

迭代器协议要求实现 _____ 和 _____ 两个方法。

答案 __enter__ ， __exit__

解析 上下文管理器协议。
示例：

```
class MyContext:
    def __enter__(self):
```

```
        print("进入上下文")
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        print("退出上下文")
        return False
# 不抑制异常
with MyContext() as ctx:
    print("上下文内部")
# 输出:
# 进入上下文
# 上下文内部
# 退出上下文
```

知识拓展： __exit__ 返回True可以抑制异常。

48 • 填空 / 简答

Pandas中，_____ 用于时间序列重采样，_____ 用于时间窗口计算。

答案 merge() ， join()

解析 Pandas数据合并方法。

示例：

```
import pandas as pd
df1 = pd.DataFrame({'key': ['A', 'B'], 'val1': [1, 2]})
df2 = pd.DataFrame({'key': ['A', 'C'], 'val2': [3, 4]})
# merge
result = pd.merge(df1, df2, on='key', how='inner')
print(result)
# join (基于索引)
df1_indexed = df1.set_index('key')
df2_indexed = df2.set_index('key')
result = df1_indexed.join(df2_indexed, how='outer')
print(result)
```

知识拓展： merge 更灵活， join 基于索引。

49 • 填空 / 简答

使用 _____ 可以创建默认字典，使用 _____ 可以创建有序字典。

答案 itertools ， chain()

解析 迭代工具模块。

示例：

```
from itertools import chain, cycle, repeat
# chain - 连接多个迭代器
list1 = [1, 2, 3]
list2 = [4, 5, 6]
for item in chain(list1, list2):
    print(item)
# 1, 2, 3, 4, 5, 6
# cycle - 无限循环
counter = 0
for item in cycle([1, 2, 3]):
    print(item)
    counter += 1
```

```
        if counter > 5:
            break
    # repeat - 重复元素
    for item in repeat('hello', 3):
        print(item)
    # hello hello hello
```

知识拓展：itertools提供高效的迭代工具。

50 • 填空 / 简答

pathlib.Path 对象的 _____ 方法用于创建符号链接，_____ 方法用于读取链接目标。

答案 with_suffix() ， with_name()

解析 Path对象修改文件名方法。

示例：

```
from pathlib import Path
p = Path('/home/user/docs/file.txt')
# with_suffix - 更改扩展名
new_p = p.with_suffix('.md')
print(new_p)
# /home/user/docs/file.md
# with_name - 更改文件名
new_p = p.with_name('newfile.txt')
print(new_p)
# /home/user/docs/newfile.txt
# 链式调用
new_p = p.with_name('report').with_suffix('.pdf')
print(new_p)
# /home/user/docs/report.pdf
```

知识拓展：这些方法返回新Path对象，不修改原对象。

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class Singleton(type):
    _instances = {}
    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]
class Database(metaclass=Singleton):
    def __init__(self):
        self.connection = "Connected"
db1 = Database()
db2 = Database()
print(db1 is db2)
print(Database._instances)
```

问题：

- (1) 第一个 print 输出什么？
- (2) 这段代码使用了什么设计模式？
- (3) 元类的作用是什么？

答案 元类单例

- (1) 输出： True
- (2) 单例模式，使用元类实现
- (3) 元类控制类的创建过程，这里用于确保只创建一个实例

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
from contextlib import contextmanager
@contextmanager
def file_manager(filename, mode):
    print(f"打开文件: {filename}")
    f = open(filename, mode)
    try:
        yield f
    finally:
        print(f"关闭文件: {filename}")
        f.close()
with file_manager('test.txt', 'w') as f:
    f.write('Hello World')
```

问题：

- (1) 这段代码的输出是什么？
- (2) @contextmanager 装饰器的作用是什么？
- (3) yield 在这里起什么作用？

答案 • (1) 输出：

```
    打开文件: test.txt
    关闭文件: test.txt
```

- (2) @contextmanager 简化上下文管理器的创建，不需要定义类
- (3) yield 分隔 __enter__ 和 __exit__ ，yield的值传给 as 后的变量

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50]
})
result = df.rolling(window=2).sum()
print(result)
print(result.iloc[-1])
```

问题：

- (1) rolling(window=2) 的作用是什么？
- (2) 第一行的结果是什么？
- (3) 最后一行的结果是什么？

答案 滚动窗口

- (1) rolling(window=2) 创建大小为2的滚动窗口
- (2) 第一行是NaN（窗口不足）
- (3) 最后一行是 A: 9, B: 90 （4+5=9, 40+50=90）

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
from pathlib import Path
from collections import defaultdict
def group_files_by_extension(directory):
    path = Path(directory)
    groups = defaultdict(list)
    for file in path.rglob('*'):
        if file.is_file():
            groups[file.suffix].append(file.name)
    return groups
result = group_files_by_extension('.')
for ext, files in result.items():
    print(f"{ext}: {len(files)} 个文件")
```

问题：

- (1) 这个函数的功能是什么？
- (2) defaultdict(list) 的作用是什么？
- (3) rglob('*') 和 glob('*') 的区别是什么？

答案 文件分组

- (1) 按文件扩展名分组统计文件
- (2) defaultdict(list) 自动为新键创建空列表
- (3) rglob() 递归搜索所有子目录， glob() 只搜索当前目录

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
from functools import reduce
from operator import add
numbers = [1, 2, 3, 4, 5]
# 方法1
result1 = reduce(add, numbers)
# 方法2
result2 = reduce(lambda x, y: x + y, numbers)
# 方法3
result3 = sum(numbers)
```

```
print(result1 == result2 == result3)
```

问题：

- (1) 输出是什么？
- (2) 三种方法有什么区别？
- (3) 哪种方法最推荐使用？为什么？

答案 函数

- (1) 输出： True
- (2) 方法1使用operator.add，方法2使用lambda，方法3使用内置sum
- (3) 方法3最推荐，因为sum是内置函数，最快且最清晰

解析

五、程序设计题

56 • 填空 / 简答

设计一个任务调度系统，要求：

Task类：

类：

- 属性：任务名、优先级、状态（待执行/执行中/已完成）
- 方法： execute() （抽象方法）
- 实现 __lt__() 用于优先级比较

Scheduler类：

类：

- 使用优先队列管理任务
- 方法：

add_task(task) ：添加任务

run_next() ：执行优先级最高的任务

run_all() ：执行所有任务

使用生成器 pending_tasks() 返回待执行任务

装饰器 @retry_on_failure(max_retries) ：

- 任务失败时自动重试
- 记录重试次数

要求：

- 实现至少2种具体任务类
- 使用 heapq 模块实现优先队列
- 添加日志记录

答案

```
import heapq
from abc import ABC, abstractmethod
from enum import Enum
import logging
from functools import wraps
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
class TaskStatus(Enum):
    PENDING = "待执行"
    RUNNING = "执行中"
```

```
    COMPLETED = "已完成"
    FAILED = "失败"
def retry_on_failure(max_retries=3):
    def decorator(func):
        @wraps(func)
        def wrapper(self, *args, **kwargs):
            retries = 0
            while retries < max_retries:
                try:
                return func(self, *args, **kwargs)
                except Exception as e:
                retries += 1
                logger.warning(f"任务{self.name} 执行失败 (尝试{retries}/{max_retries}
                if retries >= max_retries:
                self.status = TaskStatus.FAILED
                raise
            return None
        return wrapper
    return decorator
class Task(ABC):
    def __init__(self, name, priority=0):
        self.name = name
        self.priority = priority
        self.status = TaskStatus.PENDING
    @abstractmethod
    def execute(self):
        pass
    def __lt__(self, other):
        return self.priority > other.priority
    def __repr__(self):
        return f"Task({self.name}, priority={self.priority}, status={self.status.value})"
class DataProcessTask(Task):
    def __init__(self, name, data, priority=0):
        super().__init__(name, priority)
        self.data = data
    @retry_on_failure(max_retries=3)
    def execute(self):
        logger.info(f"执行数据处理任务: {self.name}")
        self.status = TaskStatus.RUNNING
        # 模拟数据处理
        result = sum(self.data)
        self.status = TaskStatus.COMPLETED
        logger.info(f"任务{self.name} 完成，结果: {result}")
        return result
class EmailTask(Task):
    def __init__(self, name, recipient, message, priority=0):
        super().__init__(name, priority)
        self.recipient = recipient
```

```
        self.message = message
    @retry_on_failure(max_retries=2)
    def execute(self):
        logger.info(f"执行邮件任务: {self.name}")
        self.status = TaskStatus.RUNNING
        # 模拟发送邮件
        logger.info(f"发送邮件到{self.recipient}: {self.message}")
        self.status = TaskStatus.COMPLETED
        return True
class Scheduler:
    def __init__(self):
        self.task_queue = []
        self.completed_tasks = []
    def add_task(self, task):
        heapq.heappush(self.task_queue, task)
        logger.info(f"添加任务: {task.name} (优先级: {task.priority})")
    def run_next(self):
        if not self.task_queue:
            logger.info("没有待执行的任务")
            return None
        task = heapq.heappop(self.task_queue)
        try:
            result = task.execute()
            self.completed_tasks.append(task)
            return result
        except Exception as e:
            logger.error(f"任务执行失败: {task.name}, 错误: {e}")
            return None
    def run_all(self):
        logger.info(f"开始执行所有任务，共{len(self.task_queue)} 个")
        while self.task_queue:
            self.run_next()
        logger.info("所有任务执行完成")
    def pending_tasks(self):
        for task in self.task_queue:
            if task.status == TaskStatus.PENDING:
                yield task
    def __len__(self):
        return len(self.task_queue)
# 使用示例
scheduler = Scheduler()
# 添加任务
scheduler.add_task(DataProcessTask("处理数据1", [1, 2, 3, 4, 5], priority=1))
scheduler.add_task(EmailTask("发送报告", "user@example.com", "报告内容", priority=3))
scheduler.add_task(DataProcessTask("处理数据2", [10, 20, 30], priority=2))
print("\n待执行任务:")
for task in scheduler.pending_tasks():
    print(f"
```

```
            {task}")
print(f"\n开始调度{len(scheduler)} 个任务\n")
scheduler.run_all()
print(f"\n已完成{len(scheduler.completed_tasks)} 个任务")
```

解析

57 • 填空 / 简答

实现一个多级缓存系统：

1. LRUCache类：

类：

- 实现LRU（最近最少使用）缓存
- 使用 OrderedDict
- 方法： get(key) 、 put(key, value)
- 实现 __getitem__ 和 __setitem__

2. CacheDecorator：

- 函数结果缓存装饰器
- 支持TTL（过期时间）
- 支持缓存统计（命中率）

3. MultiLevelCache：

- 组合多个缓存层
- L1缓存（内存）、L2缓存（文件）
- 自动降级和升级

要求：

- 使用 pathlib 处理文件缓存
- 使用 pickle 序列化数据
- 实现缓存预热和清理

答案

```
from collections import OrderedDict
from pathlib import Path
import pickle
import time
from functools import wraps
class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = OrderedDict()
    def get(self, key):
        if key not in self.cache:
            return None
        self.cache.move_to_end(key)
        return self.cache[key]
    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)
    def __getitem__(self, key):
```

```
        return self.get(key)
    def __setitem__(self, key, value):
        self.put(key, value)
    def __len__(self):
        return len(self.cache)
    def __repr__(self):
        return f"LRUCache(capacity={self.capacity}, size={len(self.cache)})"
class CacheDecorator:
    def __init__(self, ttl=None):
        self.ttl = ttl
        self.cache = {}
        self.timestamps = {}
        self.hits = 0
        self.misses = 0
    def __call__(self, func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            key = str(args) + str(kwargs)
            current_time = time.time()
            # 检查缓存
            if key in self.cache:
                if self.ttl is None or (current_time - self.timestamps[key]) < self.ttl:
                self.hits += 1
                return self.cache[key]
            # 缓存未命中
            self.misses += 1
            result = func(*args, **kwargs)
            self.cache[key] = result
            self.timestamps[key] = current_time
            return result
        wrapper.cache_info = lambda: {
            'hits': self.hits,
            'misses': self.misses,
            'hit_rate': self.hits / (self.hits + self.misses) if (self.hits + self.misses)
        }
        wrapper.cache_clear = lambda: self.cache.clear()
        return wrapper
class MultiLevelCache:
    def __init__(self, l1_capacity=100, cache_dir='cache'):
        self.l1_cache = LRUCache(l1_capacity)
        self.cache_dir = Path(cache_dir)
        self.cache_dir.mkdir(exist_ok=True)
    def get(self, key):
        # 先查L1缓存
        value = self.l1_cache.get(key)
        if value is not None:
            print(f"L1缓存命中: {key}")
            return value
```

```
        # 查L2缓存（文件）
        cache_file = self.cache_dir / f"{key}.pkl"
        if cache_file.exists():
            print(f"L2缓存命中: {key}")
            with cache_file.open('rb') as f:
                value = pickle.load(f)
            # 升级到L1
            self.l1_cache.put(key, value)
            return value
        print(f"缓存未命中: {key}")
        return None
    def put(self, key, value):
        # 存入L1
        self.l1_cache.put(key, value)
        # 存入L2
        cache_file = self.cache_dir / f"{key}.pkl"
        with cache_file.open('wb') as f:
            pickle.dump(value, f)
    def clear(self):
        self.l1_cache.cache.clear()
        for file in self.cache_dir.glob('*.pkl'):
            file.unlink()
# 使用示例
print("=== LRU缓存示例 ===")
lru = LRUCache(3)
lru['a'] = 1
lru['b'] = 2
lru['c'] = 3
print(f"缓存: {dict(lru.cache)}")
lru['d'] = 4
            # 'a'被淘汰
print(f"添加'd'后: {dict(lru.cache)}")
print("\n=== 缓存装饰器示例 ===")
@CacheDecorator(ttl=5)
def expensive_function(n):
    print(f"计算{n}...")
    time.sleep(0.1)
    return n ** 2
print(expensive_function(5))
# 从缓存获取
print(f"缓存统计: {expensive_function.cache_info()}")
print("\n=== 多级缓存示例 ===")
ml_cache = MultiLevelCache(l1_capacity=2)
ml_cache.put('key1', 'value1')
ml_cache.put('key2', 'value2')
ml_cache.put('key3', 'value3')
print(ml_cache.get('key1'))
# L2命中，升级到L1
```

```
print(ml_cache.get('key1'))
# L1命中
```

解析

58 • 填空 / 简答

使用Pandas创建自动化数据分析报告：
给定销售数据（日期、区域、产品、销量、金额）：
1. 数据清洗：
• 处理缺失值和异常值
• 数据类型转换
• 去重
2. 统计分析：
• 描述性统计
• 按维度聚合（区域、产品、时间）
• 同比环比分析
• 趋势分析（移动平均、增长率）
3. 数据可视化：
• 销售趋势图
• 产品占比饼图
• 区域对比柱状图
4. 报告生成：
• 导出Excel（多个sheet）
• 生成HTML报告
• 添加图表和表格
要求：
• 使用Pandas的高级功能
• 使用matplotlib/seaborn绘图
• 代码模块化

答案

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from datetime import datetime, timedelta
class SalesReportGenerator:
    def __init__(self, data):
        self.df = data.copy()
        self.cleaned_df = None
        self.analysis_results = {}
    def clean_data(self):
        print("开始数据清洗...")
        # 处理缺失值
        self.df.dropna(subset=['日期', '产品'], inplace=True)
        self.df['销量'].fillna(self.df['销量'].median(), inplace=True)
        self.df['金额'].fillna(self.df['金额'].median(), inplace=True)
        # 数据类型转换
        self.df['日期'] = pd.to_datetime(self.df['日期'])
        self.df['销量'] = self.df['销量'].astype(int)
```

```
        self.df['金额'] = self.df['金额'].astype(float)
        # 去重
        self.df.drop_duplicates(inplace=True)
        # 处理异常值（使用IQR方法）
        Q1 = self.df['金额'].quantile(0.25)
        Q3 = self.df['金额'].quantile(0.75)
        IQR = Q3 - Q1
        self.df = self.df[
            (self.df['金额'] >= Q1 - 1.5 * IQR) &
            (self.df['金额'] <= Q3 + 1.5 * IQR)
        ]
        self.cleaned_df = self.df.copy()
        print(f"清洗完成，剩余{len(self.cleaned_df)} 条记录")
def descriptive_statistics(self):
    print("\n描述性统计...")
    stats = self.cleaned_df[['销量', '金额']].describe()
    self.analysis_results['descriptive'] = stats
    return stats
def aggregate_analysis(self):
    print("\n聚合分析...")
    # 按区域聚合
    region_agg = self.cleaned_df.groupby('区域').agg({
        '销量': 'sum',
        '金额': 'sum'
    }).sort_values('金额', ascending=False)
    # 按产品聚合
    product_agg = self.cleaned_df.groupby('产品').agg({
        '销量': 'sum',
        '金额': 'sum'
    }).sort_values('金额', ascending=False)
    # 按时间聚合
    self.cleaned_df.set_index('日期', inplace=True)
    time_agg = self.cleaned_df.resample('W').agg({
        '销量': 'sum',
        '金额': 'sum'
    })
    self.analysis_results['region'] = region_agg
    self.analysis_results['product'] = product_agg
    self.analysis_results['time'] = time_agg
    return region_agg, product_agg, time_agg
def trend_analysis(self):
    print("\n趋势分析...")
    # 移动平均
    self.cleaned_df['MA_7'] = self.cleaned_df['金额'].rolling(window=7).mean()
    # 增长率
    self.cleaned_df['增长率'] = self.cleaned_df['金额'].pct_change() * 100
    self.analysis_results['trend'] = self.cleaned_df[['金额', 'MA_7', '增长率']]
    return self.cleaned_df[['金额', 'MA_7', '增长率']]
```

```
def create_visualizations(self):
    print("\n创建可视化...")
    fig, axes = plt.subplots(2, 2, figsize=(15, 10))
    # 销售趋势图
    self.analysis_results['time'].plot(y='金额', ax=axes[0, 0], title='销售趋势')
    axes[0, 0].set_ylabel('金额')
    # 产品占比饼图
    product_sales = self.analysis_results['product']['金额']
    axes[0, 1].pie(product_sales, labels=product_sales.index, autopct='%1.1f%%')
    axes[0, 1].set_title('产品销售占比')
    # 区域对比柱状图
    self.analysis_results['region']['金额'].plot(kind='bar', ax=axes[1, 0], title='区域
    axes[1, 0].set_ylabel('金额')
    axes[1, 0].tick_params(axis='x', rotation=45)
    # 移动平均图
    self.cleaned_df[['金额', 'MA_7']].plot(ax=axes[1, 1], title='销售额与移动平均')
    plt.tight_layout()
    plt.savefig('sales_analysis.png')
    print("图表已保存: sales_analysis.png")
def generate_report(self, filename='sales_report.xlsx'):
    print(f"\n生成报告: {filename}")
    with pd.ExcelWriter(filename, engine='openpyxl') as writer:
        # 原始数据
        self.cleaned_df.to_excel(writer, sheet_name='清洗后数据')
        # 描述性统计
            self.analysis_results['descriptive'].to_excel(writer, sheet_name='描述性统计')
            # 区域分析
            self.analysis_results['region'].to_excel(writer, sheet_name='区域分析')
            # 产品分析
            self.analysis_results['product'].to_excel(writer, sheet_name='产品分析')
            # 时间序列
            self.analysis_results['time'].to_excel(writer, sheet_name='时间序列')
        print(f"报告已生成: {filename}")
# 使用示例
np.random.seed(42)
dates = pd.date_range('2024-01-01', periods=90, freq='D')
regions = ['华东', '华南', '华北', '西南']
products = ['产品A', '产品B', '产品C', '产品D']
data = {
    '日期': np.random.choice(dates, 500),
    '区域': np.random.choice(regions, 500),
    '产品': np.random.choice(products, 500),
    '销量': np.random.randint(10, 200, 500),
    '金额': np.random.uniform(1000, 10000, 500)
}
df = pd.DataFrame(data)
# 生成报告
generator = SalesReportGenerator(df)
```

```
generator.clean_data()
generator.descriptive_statistics()
generator.aggregate_analysis()
generator.trend_analysis()
generator.create_visualizations()
generator.generate_report()
print("\n分析完成！")
```

题目4和题目

和题目5的参考答案由于篇幅限制，这里提供核心实现思路：

解析

59 • 填空 / 简答

编写一个文件同步工具：

1. FileSyncer类：

类：

- 比较源目录和目标目录
- 识别新增、修改、删除的文件
- 执行同步操作

2. 功能要求：

- 支持增量同步
- 支持文件过滤（按扩展名、大小）
- 支持双向同步
- 冲突处理策略

3. 使用生成器：

- scan_directory() ：扫描目录
- find_differences() ：查找差异
- sync_files() ：同步文件

要求：

- 使用 pathlib 进行所有文件操作
- 使用哈希值检测文件变化
- 添加进度显示
- 支持断点续传

答案 ：文件同步工具- 核心是使用 pathlib 遍历目录，计算文件哈希值比较差异，使用生成器逐个处理文件。

解析

60 • 填空 / 简答

实现一个可扩展的插件系统：

1. Plugin基类：

基类：

- 抽象方法： initialize() 、 execute() 、 cleanup()
- 属性：名称、版本、描述

答案 ：插件系统框架- 核心是使用元类或装饰器实现插件自动注册，使用 importlib 动态加载模块，实现钩子系统管理插件生命周期。

评分说明：

- 单选题：每题1分，共30分

- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析